

Fire Detection

Детекция открытого огня — YOLO11s

Сероглазов Андрей Александрович · группа 6-ИФСТ-11 · индивидуальная работа

Существующие решения

Источник	Архитектура	Ключевой результат
MDPI Fire	YOLOv5/v7/v8	Высокая скорость инференса, приемлемая точность
Sensors	YOLOX-L + Defogging	mAP 87.47%, устойчивость к задымлению
ForestGuard	YOLOv8 + Federated	55–73 мс/кадр на Jetson Nano / Raspberry Pi
F3-YOLO	YOLOv12n + area attention	Лучшая детекция мелких/перекрытых очагов
PMC	YOLO11n + EMA-внимание	Подтверждает применимость YOLO11 к задаче

 **Вывод:** YOLO11 — обоснованный выбор: баланс скорости/точности, готовая экосистема Ultralytics.

Анализ датасетов

DBA-Fire

Класс **fire** · базовый набор изображений огня

fireANDsmoke

Классы **fire + smoke** · разнообразие сцен

FASDD

Классы **fire + smoke** · крупный датасет, разные условия съёмки

Собственные кадры из видео

Класс **fire** · реальные возгорания, целевой сценарий

DBA-Fire (БАЗОВЫЙ)



fireANDsmoke (СМЕШАННЫЙ)



FASDD (КРУПНЫЙ, РАЗНЫЕ УСЛОВИЯ)



СОБСТВЕННЫЕ ВИДЕОКАДРЫ (РЕАЛЬНЫЕ)



**ВСЕ ДАННЫЕ БЕЗ АУГМЕНТАЦИИ,
РАЗМЕТКА ПРОВЕРЕНА ВРУЧНУЮ.**

Сбор и разметка данных

≈800 train / ≈210 val

Требование выполнено:
≥800 / ≥200

fire_dataset_viewer.py

Python + tkinter:
одобрение/отклонение
одним нажатием,
авторазбивка train/val,
статистика сцен



Найденная проблема: при переименовании класса "0" → "fire" в Roboflow старые аннотации не пересчитались — образовалось два обозначения одного объекта.

Исправлено объединением класса на Roboflow с генерацией новой версии датасета.

Roboflow

Разметка bbox: открытые датасеты — проверка и коррекция; видеокадры — разметка с нуля

Используемые аугментации

Параметр	Значение	Обоснование
degrees	10	Имитация наклона камеры
translate	0.1	Огонь редко в центре кадра
scale	0.5	Очаг может быть мелким или во весь кадр
fliplr / flipud	0.5 / 0.1	fliplr оправдан; flipud спорно — создаёт «перевернутый огонь»
mosaic	1.0	Помогает видеть мелкие очаги; работал все 150 эпох
mixup	0.15	Спорно для полупрозрачной текстуры пламени
hsv_s / hsv_v	0.7 / 0.4	Агрессивно — ради разных условий освещения
dropout / weight_decay	0.1 / 0.0005	Регуляризация при небольшом датасете

Параметры обучения

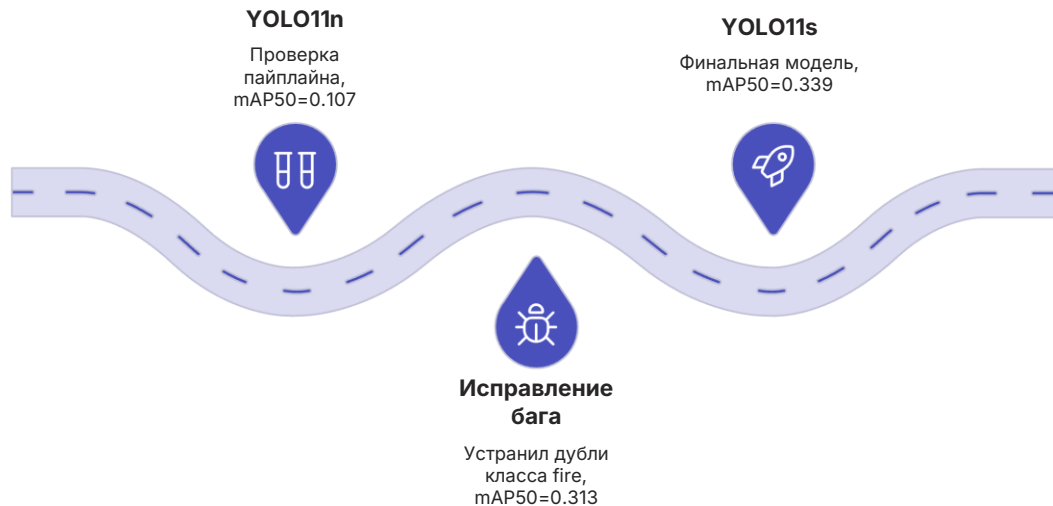
Среда

Google Colab · GPU Tesla T4

Итоговые параметры YOLO11s

Обучение прошло все 150 эпох без early stopping.

Путь к финальной модели



epochs = 150 · **imgsz** = 640 · **batch** = 16

lr0 = 0.01 · **lrf** = 0.01 · **cos_lr** = True · **patience** = 30

Результаты работы модели

0.339

mAP50
Финальная YOLO11s

0.127

mAP50-95

0.47

Precision

0.39

Recall

0.393

AP (fire)



Анализ ошибок

Confusion Matrix

~250 ложных срабатываний (фон→fire)

~252 пропуска (fire→фон)

Граница классов не выработана уверенно — особенно на тёплом освещении (кожа у костра, закат).



Ключевой вывод: ограничивающий фактор — объём и разнообразие данных, прежде всего нехватка отрицательных примеров.

Тип ошибки	Причина
Пропуск синего пламени	Цвет вне обучающей выборки
Ложные срабатывания	Небо, фары автомобилей
Пропуск при перекрытии	Объект закрывает очаг
Неточная локализация	Неполный bbox
Пропуск мелких очагов	Малый размер объекта
Нестабильность на видео	Флуктуации между кадрами
Неразделение соседних очагов	Заполненное огнём пространство

ДЕМОНСТРАЦИЯ

Демонстрация работы

Приложение **main.py**: выбор видеофайла → покадровое чтение OpenCV → инференс YOLO11s → отрисовка bbox в реальном времени.



compressed_compressed_zapis_2026-07-19_183752.mp4



Ограничения и развитие

Текущие ограничения

Технологии

YOLO11s · Ultralytics · Roboflow · Google Colab (Tesla T4) · OpenCV · Tkinter · Python 3.12

Направления развития

→ Отрицательные примеры

Закат, лица у костра, фары

→ Плотная разметка

Точнее обозначать границы пламени

→ Увеличение `imgsz`

Лучше детектировать мелкие очаги

→ YOLO11m

При пропорциональном росте датасета

Датасет ~800/200 изображений

Умеренный Recall (0.28–0.43)

Путаница с тёплыми/яркими объектами